

Human and machine collaboration for painting game assets with deep learning

Ygor Rebouças Serpa^{a,*}, Maria Andréia Formico Rodrigues^b

^a Onanim Game Studio, Fortaleza, Brazil

^b Programa de Pós-Graduação em Informática Aplicada (PPGIA), Universidade de Fortaleza (UNIFOR), Fortaleza, Brazil

ARTICLE INFO

Keywords:

Machine learning
Deep learning
Procedural content generation
Image generation
Games

ABSTRACT

Modern games are among the most intricate pieces of software ever devised. However, artists have little to no means of porting their creative work from title to title as the aesthetics change and older models grow to look obsolete. Zooming into the topic of assisted game art generation, the literature is notably scarce, although advances towards automated asset generation are of paramount interest to the field. In this work, we investigate the use of deep learning algorithms to create pixel art sprites from line art sketches to produce artwork of sufficient quality to be used within a game product with little to no manual editing by human artists. Such a problem contrasts with well-known tasks studied in the literature, which are based on natural pictures, boast massive datasets, and are much more tolerant to noise. In addition, we conducted a case study of applying current technology to the drawing pipeline of an upcoming game title, attaining useful and positive results that may fast-track the game development, supporting the argument that current image generation state-of-the-art is ready to be used in some real-world tasks.

1. Introduction

Designing games is a profound technical and artistic challenge [1]. Modern games are among the most intricate pieces of software ever devised, touching almost every field of computer science while featuring thousands of intricate models, high-resolution textures, skeletal animations, and complex soundtracks [2]. To cope with the ever-growing complexity, developers decouple the needed infrastructure as an “engine”, which can be developed independently and shared among several titles [3]. On the other hand, artists have little to no means of porting their work from title to title, as the aesthetics change and older models grow to look obsolete. In this regard, advances towards assisted or automated asset generation are of paramount interest to the field.

Artistic work, within games, range from conceptualizing the game and its mechanics to painting textures and sprites, creating visual models and their animations, composing the soundtrack, and writing down dialogues. Collectively, these elements are known as the game’s content, and the automation of such tasks is known as Procedural Content Generation (PCG) [4–6]. This field draws methods from a wide range of disciplines, such as graph theory, image processing, physics, AI, and natural language processing. A recent survey on the topic is

presented by Liu et al. [7].

Since 2012, with the debut of AlexNet [8], the computer vision community strode greatly in how well computers can parse and generate images. For the first time, computers became able to generate realistically looking images out of nowhere [9–11]. Critically, we wonder: *Is the current state-of-the-art in image generation usable to generate 2D content in lieu of human artists?* Or, more generally, can vision models be effectively used to reduce iteration times for asset production?

To explore this inquiry, we considered *Trajes Fatais: Suits of Fate*, a 2D pixel-art fighting game inspired by the late nineties classics of the genre. In particular, we investigate drawing character sprites from rough line-art sketches using neural networks [12]. Concretely, each character has an average of 508 sprites, which are drawn as two independent sub-sprites, one dedicated to shading and the other to coloring, as shown in Fig. 1. Using deep learning [13], we train models to translate line-art sketches into each of these sub-sprites, striving to generate content that can be polished/accepted by a human artist in less time than it would take to draw the asset from scratch.

This work extends our previous inquiry on the topic, which evaluated the Pix2Pix [14] architecture as a means to translate sketches into usable artwork [15], obtaining moderately successful results. In summary,

* Corresponding author.

E-mail address: ygor.reboucas@gmail.com (Y. Rebouças Serpa).

<https://doi.org/10.1016/j.entcom.2022.100497>

Received 8 May 2021; Received in revised form 10 April 2022; Accepted 23 April 2022

Available online 9 May 2022

1875-9521/© 2022 Elsevier B.V. All rights reserved.

regressing shading values worked remarkably well, outputting similar results to the human-drawn sprites, while coloring data could not be adequately inferred by the algorithm, yielding noisy/blurred images. Additionally, the algorithm performed poorly on challenging poses for both tasks and, especially for the coloring problem, quantizing the continuous color range to the set of allowed colors introduced significant artifacts.

In this paper, we propose a radically different approach based on casting the problem as a per-pixel classification task using a combination of the U-Net architecture [16,17], dense connections [18], focal loss [19], class weighting [20] and multi-level supervision [21], while removing the discriminator and adversarial training components [14]. In our experiments, we obtained vastly improved results for both shading and coloring problems while solving many of the shortcomings of the previous implementation. Also, qualitatively, the game's art team reported that many of the new generated sprites are almost ready to be used on real game production, with little to no manual editing. Code and data available at <https://github.com/nuzrub/PaintingGameAssetsWithDeepLearning>.

2. Related work

Since the debut of GANs in 2014 [10], the image generation field has made significant advances. In its earlier years, authors tackled stable training and mode collapse [22–24]. By 2017, realistically looking high-resolution portraits could be generated [25], soon followed by GANs that can reproduce most classes in ImageNet [26] and models that allow some degree of artistic control [11,27,28]. Closely related, models for style transfer [29], image colorization [30], domain translation [14,31], super-resolution [32,33] and image interpolation were also proposed [34,35]. In parallel, while these works address natural images, several authors have devoted their time to other visual styles, such as cartoon [36,37] and anime images [38,39].

While GANs dominate most of the earlier literature, other approaches have gained traction in later years. Most noticeably, variational auto-encoders (VAEs) [9,40] and auto-regressive models [41–43]. While GANs avoid a direct probabilistic formulation, VAEs explicitly model the dataset probability distribution and learn to sample it to generate new images. In turn, auto-regressive models generate images one pixel at a time by sampling a learned probability distribution and conditioning it to all previously generated pixels. Besides formulation innovations, many authors are exploring the use of Attention/Transformer based architectures within the image generation context [44–46].

At large, generative models can be divided into fully unsupervised/weakly supervised approaches, which generate images primarily from noise/labels [11,26,27,42], and strongly supervised methods, which start from an input image that strongly conditions the final output [14,30,31,34,35]. The latter is of special interest to asset generation as it allows a greater degree of artistic control. In addition, model training is considerably less resource-intensive [14] when input–output pairs are available than when there are only unpaired samples [31].

Zooming into the topic of game art generation, the literature is notably scarce [4,6,7]. To the best of our knowledge, only a few authors

have tackled the automatic sprite generation problem. Horsley et al. attempted to generate 8-bit sprites from random noise using a simple GAN model, achieving limited results [47]. Concurrently, Xue et al. sought to generate plausible next-frame sprites for a given animation sequence using a probabilistic CNN model [35]. More recently, Hong et al. proposed a multi-discriminator GAN to generate novel sprites from a set of shape, color and animation cues. Tackling the problem in the opposite direction, Smirnov et al. proposed a model to extract sprites from game scenes [48]. Beyond sprite generation, most deep learning works within PCG are dedicated to 2D or 3D level generation [5–7,49].

We attribute the scarcity of published works dealing with automatic sprite generation and, more specifically, character design, to the amount of detail involved and the importance of such assets. Poorly generated characters would be readily noticed by users, negatively impacting the overall game experience. In this context, it remains open whether current approaches are ready to solve real-world tasks, which often (1) lack large datasets, (2) deviate considerably from natural images, and (3) are extremely intolerant to errors and artifacts of any kind.

This work's main contribution to state-of-the-art image generation is a resounding yes to this open question. To reach this conclusion, we conducted a case study of applying current technology to the drawing pipeline of an upcoming game title, named *Trajes Fatais: Suits of Fate*, attaining useful results that may fast-track the game development. Such a problem contrasts with well-known tasks studied in the literature [11,26,42,50], which are based on natural pictures, boast massive datasets, and are much more tolerant to noise.

3. The sprite creation pipeline

In *Trajes Fatais: Suits of Fate*, the art director creates every sprite as a rough sketch to test the animation ideas. After this initial step, these sketches are redrawn as line-art mock-ups and are handed over to the drawing team to be completed. Fig. 2 shows two sprites at their early sketch stage. This pipeline centers the characters' design around the art director while delegating the creative drawing to the other artists. On average, artists take less than two minutes to draw initial sketches and no more than ten minutes to draw their line-art counterparts.

To replicate the late nineties look, each character is limited to 252 colors. To effectively use this palette in a meaningful way, each sprite is drawn twice: as a shading sprite and as a coloring one. The former uses up to 6 gray tones to convey the shading information and, the latter, up to 42 different colors to mark the different regions of the sprite, such as hair, skin, clothes, accessories, etc. Combined, these two sprites identify

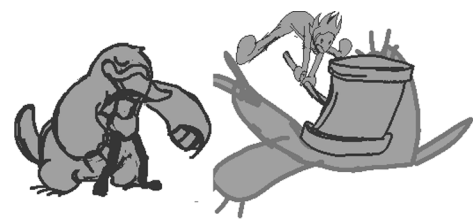


Fig. 2. Two Sarah sprites in their early sketch stages.



Fig. 1. Stages of a *Trajes Fatais: Suits of Fate* sprite. From left to right, the initial line-art sketch, the shading sprite, the coloring sprite, the shading & color sprites encoded together, and the final in-game sprite, using the character's default palette.

up to $6 * 42 = 252$ unique areas, which are encoded as a final sprite. In game, each shading-color pair is assigned a color from a palette, creating the character's final look, as shown in Fig. 1. This procedure streamlines the artist's work as it relaxes the 252 color limitation into two distinct and intuitive tasks. Fig. 3 shows the shading and coloring sprites of three characters in their idle pose.

Of all stages, drawing shading sprites is the most time-consuming step, taking anywhere from 30 to 90 min to be completed, as shading complexity varies significantly across characters due to clothing, hair, and muscles. On the other hand, drawing the color sprites is considerably easier, as the task is mainly to label each region with its associated color. These sprites take at most 15 min to be completed, with most of this time being devoted to fine features, such as the eyes, lips, and accessories.

Finally, it is noteworthy to mention that, although 42 individual coloring tones can be used, most characters employ less than the maximum set of tones. On average, each character uses 37 tones, with the least using only 24. Moreover, some of these tones are assigned to effects, never being used on the character itself.

4. Deep-learning asset generation

The problem of converting line-art sketches into shading and coloring sprites is essentially a semantic segmentation problem, in which every pixel has to be labeled among one of 6 shadings and 42 coloring tones, respectively. In our previous work, we approached the problem using the Pix2Pix architecture [14], which learns to regress each pixel to the target tone under an adversarial learning setting [10]. Among the shortcomings of this approach, the algorithm is biased towards the average tone, and the final output has to be quantized to the set of allowed colors.

On top of the aforementioned issues, we also highlight that both tasks have class imbalance issues. On shading sprites, the sixth tone (the highlight) is sparingly used in relation to the others. On coloring sprites, labels such as the eyes and lips occupy a minor fraction of the entire sprite while others such as the legs and arms dominate most of the image. For that reason, techniques to balance each tone's importance are of paramount significance to this problem.

Finally, it is noteworthy to mention that, artistically, not all errors are valued the same. For instance, mistaking the sixth shading tone (highlight) with the fifth (bright) is less of an error than mistaking it with the first (darkest tone). Likewise, painting the eye region with the face tone is more acceptable than painting it with the leg color.

To address each of these issues, we combined several deep learning techniques into a custom-tailored model. First, we base ourselves on the U-Net architecture [16], which is the encoder-decoder model that powers most semantic segmentation networks. Second, we elaborate the downsampling and upsampling steps using dense connections [18], similarly to the Multi-Resolution U-Net model [17]. Third, in a similar style to the U-Net 3 + network [21], we employ deep supervision to regularize our model. And, finally, regarding our loss function, we employ the focal loss function [19] weighted by the effective number of samples of each class [20], addressing the class imbalance issues. Fig. 4 shows an overview of all these elements. In the following subsections, We elaborate on each of these elements.

4.1. U-Net architecture

Our network design originates from the famous U-Net architecture, which consists of an encoder-decoder model with skip connections between the encoder and decoder layers [16,51]. The encoder is a neural network that progressively down-samples the input while increasing the number of dimensions, effectively converting spatial information into semantic features. In the opposite direction, the decoder distills the learned semantics back into the spatial domain, progressively up-sampling the input while lowering the number of dimensions. In-between, the output of each encoder step is also fed to the respective decoder step, creating a skip-connection.

Among these elements, the skip-connection is the highlight of this architecture. In practice, connecting the decoder steps with the raw encoder outputs allows the network to recall low-level features directly, such as shapes and edges, instead of encoding these elements along with the semantic features. Additionally, it greatly improves the gradient flow within the model, as gradients can flow uninterrupted from one side to the other.

In our case, we developed the architecture to input and output 256×256 pixels images in one-hot encoding [51] using 7 down/up-sampling steps. Input line-art sketches have 3 possible labels: background, silhouette, and fill; shading sprites have 7 possible labels, six shading tones, and a dedicated background class; and, likewise, coloring sprites can have up to 42 color labels plus an additional background class.

4.2. Dense connections

The original U-Net model employs two convolution operations per down-sampling step and three for each up-sampling one. To allow the model to explore the input space further, more layers are needed. However, this can quickly escalate to prohibitive proportions. To balance network depth with resource usage, we resort to dense connections [18] similarly to the work of Ibtehaz and Rahman [17].

At a high level, given a convolution with f filters, a dense block factors this operation into n sequential convolutions with f/n filters. Then, the outputs of each intermediate convolution are concatenated to compose a tensor with f channels. Finally, to improve gradient flow, a residual connection is added, summing the original input with the final concatenated features, using a 1×1 convolution to equate the number of dimensions, if necessary. This process is shown in Fig. 5.

In practice, this also reduces the number of parameters used by the network. While a single $s \times s$ convolution operating over f input filters and generating f output filters requires $p = s^2 f^2$ parameters, the respective number of parameters used by its dense block replacement is given as:

$$p = s^2 \left(\frac{f^2}{n} + \left(n - 1 \right) \frac{f^2}{n^2} \right)$$

The first term corresponds to the first convolution downsizing the f filters into f/n channels and the remaining $n - 1$ convolutions operating at the f/n scale. In this work, we use $n = 3$, thus, the previous equation boils down to $p = \frac{4}{3} s^2 f^2$, slightly less than half the original number of parameters. For this reason, we use three and five dense blocks for each down/up-sampling step, respectively.



Fig. 3. From left to right, examples of Lucy, Sarah, and Saulo. Shading and color sprites.

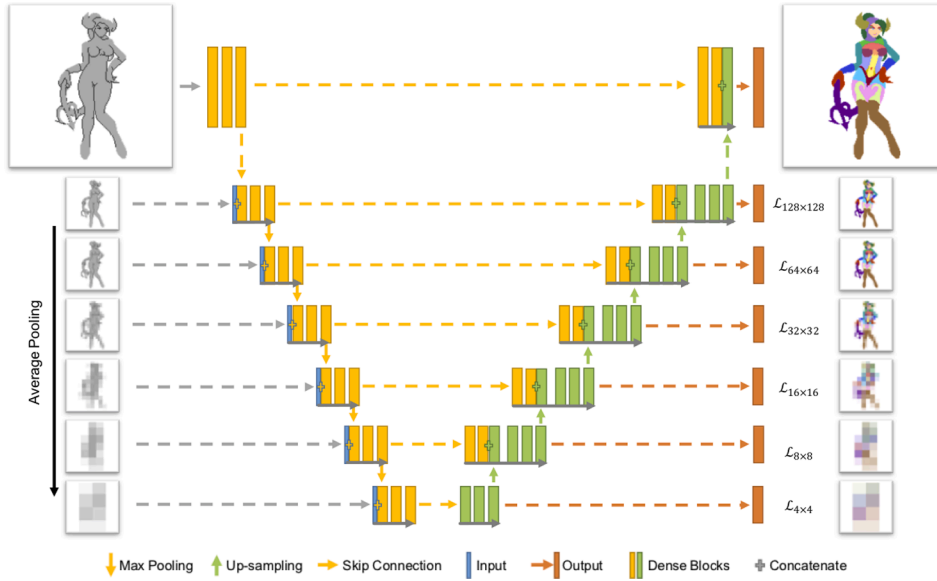


Fig. 4. Overview of our solution's architecture. Encoder/Decoder layers are in yellow and green, respectively, while the dense supervision is in blue (inputs) and orange (outputs).

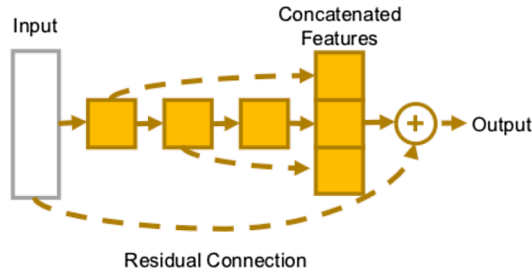


Fig. 5. Internal layout of a dense block. Instead of using a single large convolution, several smaller ones are used and their outputs are stacked to create the intended tensor. Finally, a residual connection is added to improve the gradient flow.

4.3. Dense supervision

Traditionally, the input image is used solely as the first layer of the network and only the last layer's results are compared to the ground truth. Dense supervision breaks this tradition by appending a down-sampled version of the input at each down-sampling step and supervising the output of each up-sampling step using a down-sampled version of the output, effectively supervising the model across every scale [21].

In our work, the benefit of using dense supervision is the automatic creation of smooth labels to regularize the network. Since we use one-hot encoding at the input and output level, this forces the algorithm to understand which tones are more prevalent within each region and which tones go together more often. Ultimately, this greatly reduces the number of blatant errors, such as mistaking the eye label for the leg label, since these two tones rarely appear around the same areas.

Care must be taken, however, to ensure the algorithm will prioritize the last layer, which is the output we are really interested in. For that, we weight the seven computed losses ($\mathcal{L}$) independently, while ensuring the total weight distribution averages to one. Concretely, the highest resolution output is given half of the total weight and each subsequent output is given half of the remainder, as shown below:

$$\mathcal{L} = 7 \left(\frac{\mathcal{L}_{256 \times 256}}{2} + \frac{\mathcal{L}_{128 \times 128}}{2^2} + \frac{\mathcal{L}_{64 \times 64}}{2^3} + \frac{\mathcal{L}_{32 \times 32}}{2^4} + \frac{\mathcal{L}_{16 \times 16}}{2^5} + \frac{\mathcal{L}_{8 \times 8}}{2^6} + \frac{\mathcal{L}_{4 \times 4}}{2^6} \right)$$

4.4. Class-weighted focal loss

Given the task of classifying each pixel as one of many labels, we follow the traditional choice of using the softmax activation in each output layer along with the categorical cross-entropy loss [51]. To improve performance, we augment this loss with a focal term [19], which down-weights each pixel proportionally to the γ -power of the algorithm confidence. Therefore, this term gradually shifts the algorithm's attention from well-classified pixels to yet unsure ones. This also counteracts class imbalance, as majority classes tend to be learned first and, as the training progresses, are gradually down-weighted, shifting the algorithm's attention to the minority classes.

To further cope with imbalance issues, we also include class weights based on the effective number of samples of each class [20]. The rationale for this term is that the net benefit of more samples diminishes as more samples are added, thus, this phenomenon must be accounted for when weighting classes to avoid over-penalization. The formula for the effective number of samples of a class with n samples is given as $(1 - \beta^n)/(1 - \beta)$, with $\beta \in [0, 1]$. To include this term in the loss formulation, the class weights are normalized, so they average to one, and their inverse is multiplied with the loss term.

Some severely underrepresented labels were assigned large weights in our specific case, pushing all other label weights down due to normalization. This had the adverse effect of slowing down training, as these labels appear infrequently. To counter this effect, we artificially added $k = 32.768$ to n when calculating the effective number of samples of each class, smoothing weights more evenly among all labels.

Putting it all together, our complete loss formulation is given as the sum of the weighted focal cross-entropy loss for each class, averaged over all N pixels. Symbolically, it is as follows:

$$\mathcal{L} = \frac{1}{N} \sum_i^C \underbrace{\frac{1 - \beta}{1 - \beta^{n_i + k}}}_{\text{class weights}} \underbrace{(1 - \hat{y}_i)^\gamma}_{\text{focal term}} \underbrace{(-y_i \log(\hat{y}_i))}_{\text{cross entropy loss}}$$

With N being the total number of pixels, C the set of classes, n the number of samples in the i^{th} class, $\beta = 0.9$ and $k = 32.768$ the class weight hyper-parameters, $\gamma = 2$ the focal factor and y and $\hat{y}$ being the ground-truth labels and the algorithm prediction, respectively.

4.5. Technical details

Regarding architecture, the model was built to work with 256×256 images, which is enough to fit most of the game sprites. For both tasks, the input has 3 channels. For shading and coloring, respectively, the output has 7 classes and up to 43 labels. Regarding deep supervision, both the input and outputs are down-sampled using 2D Average Pooling, while, within the network, down-sampling layers use the Max Pooling operation. Regarding up-sampling, we do not use transposed convolutions. Instead, we rely on the Up Sampling layer with the nearest-neighbor interpolation. Finally, all convolutions are 3×3 with stride one, followed by batch normalization [52] and the *ReLU* activation function [51]. We do not employ dropout or weight decay.

Regarding network size, each down/up-sampling stage uses $f * 72$ filters, being f equals to 1, 2, 4, 8, 8, 8, 8, when down-sampling, and f equals to 8, 8, 8, 8, 4, 2, 1, when up-sampling. In total, the final model has $X = 53,791,206$ trainable parameters and $y = 209$ convolutions.

All models were trained in mixed-precision mode using the *RMSprop* optimizer with a learning rate of 0.001 for 200 epochs in batches of 8 images. Augmentation-wise, we only used random horizontal flipping in all experiments. All experiments were done using the TensorFlow 2 Keras API [53] on Windows using a single GTX 2070 GPU. Depending on the configuration, training a complete model from beginning to end takes from one to two hours, on average. Inference times are negligible.

5. Tests and results

In this section, we discuss the results obtained with our solution on *Trajes Fatais: Suits of Fate*'s pipeline. We subdivide this section as follows: a description of the dataset and evaluation metrics, overall results, ablation study and, finally, some potential threats to validity.

5.1. Dataset description

We bring out and discuss our model results for three game characters: *Lucy*, *Sarah*, and *Saulo*. *Lucy* is an adult woman wearing collant clothing, *Sarah* is a tiny girl with a big platypus suit, and *Saulo* is a young male wearing a scarecrow costume. Unlike in our previous work [15], all three characters are now fully drawn by the human artists, having respectively 602, 352, and 588 sprites, out of which 170, 94, and 164 are reserved as the validation set. Regarding colors, *Lucy*, *Sarah*, and *Saulo* use, respectively, 28, 37, and 30 distinct coloring tones.

Among the game's fifteen characters, we selected those based on their visual features. *Lucy* sprites work well as a baseline, being easy to shade and color, with most tones being equally frequent. *Sarah* has many large smooth surfaces that are easy to shade and color. However, the girl within the suit is often shadowed by the costume and has many infrequent color tones that can be hard to infer. Finally, *Saulo* presents a shading challenge, often going from the darkest to the brightest shading tones and back in a matter of pixels. Fig. 6 shows the distribution of shading tones, from darker to brighter, and color tones, from most to least frequent.

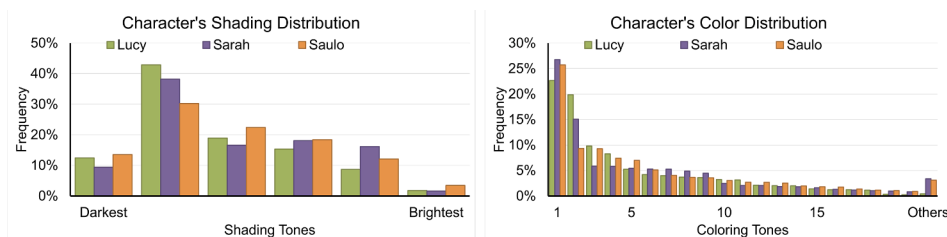


Fig. 6. Distribution of shading tones (left), from darker to brighter, and color tones (right), from most to least frequent, for all three characters.

5.2. Evaluation metrics

To assess the algorithm efficacy, we employed two main metrics: accuracy and balanced accuracy. The former measures the number of correctly predicted pixels out of the total, without correcting for class imbalance. On the other hand, the latter takes into account the number of samples in each class. It is computed as the average recall among all classes.

To evaluate the algorithm's confusion, we also computed the top- k accuracy metric, with $k = 3$ and $k = 5$. This score evaluates if the correct class is among the k classes the algorithm is most confident for the given pixel. Therefore, when compared to the raw accuracy, this metric reveals the percentage of pixels that were almost classified correctly.

Finally, after computing these metrics, we discarded the background class, which can easily dominate the results, as it is easily predictable and is by far the most frequent class. In summary, all presented results are relative to sprite pixels.

5.3. Overall results

Based on our validation set, we extracted a set of shading and coloring images generated by our algorithm, along with their respective ground truth (Fig. 7). To select these images, we converted the one-hot encoded outputs to the RGB color space and computed each sprite root-mean-squared error (RMSE) against the ground truth. Then, we sorted sprites by error, selecting six samples from each task for presentation. From bottom-to-top, the first sprite is the one with the least error and the last is the one with the most. We chose to select sprites by RMSE instead of accuracy as this score correlates well to how the human visual system perceives errors in images. Moreover, this avoids presentation bias, as the image were mechanically selected rather than chosen by hand.

Regarding shading, all three characters present very good results, with most of the predicted shading correlating to the human drawn one. Zooming into each image, most of the inaccuracies tend to occur at the edges of shading transitions and, more specifically, along the highlight (the brightest white tone). Specific to each character, most of the inaccuracies within *Lucy* sprites are at her hair and hands, around *Sarah*'s joints and clothes, and along *Saulo*'s face, torso and pants, being this last one the character with most errors. The highest error sprites (topmost row) all come from special attack animations, which usually employ weapons and challenging poses. Collectively, all these sprites, despite not being perfect, present many areas that could be used in the game as is or with little to no alteration. Moreover, many of the shading deviations are consistent with the sprite pose and could have been a human artist choice.

Regarding color sprites, most deviations occur along region boundaries and around minor elements, such as the eyes and lips, while more representative regions, like the arms, legs and clothes, tend to be correctly predicted in most instances. As we move to the highest error sprites, most inaccuracies appear around infrequent elements, such as weapons or, in the *Sarah* case, along the girl, when she leaves the platypus suit. At the topmost row, the highest error sprite of each character also come from special attack animations, that call for challenging poses and, in *Saulo*'s case, calls for his crow companion. In a

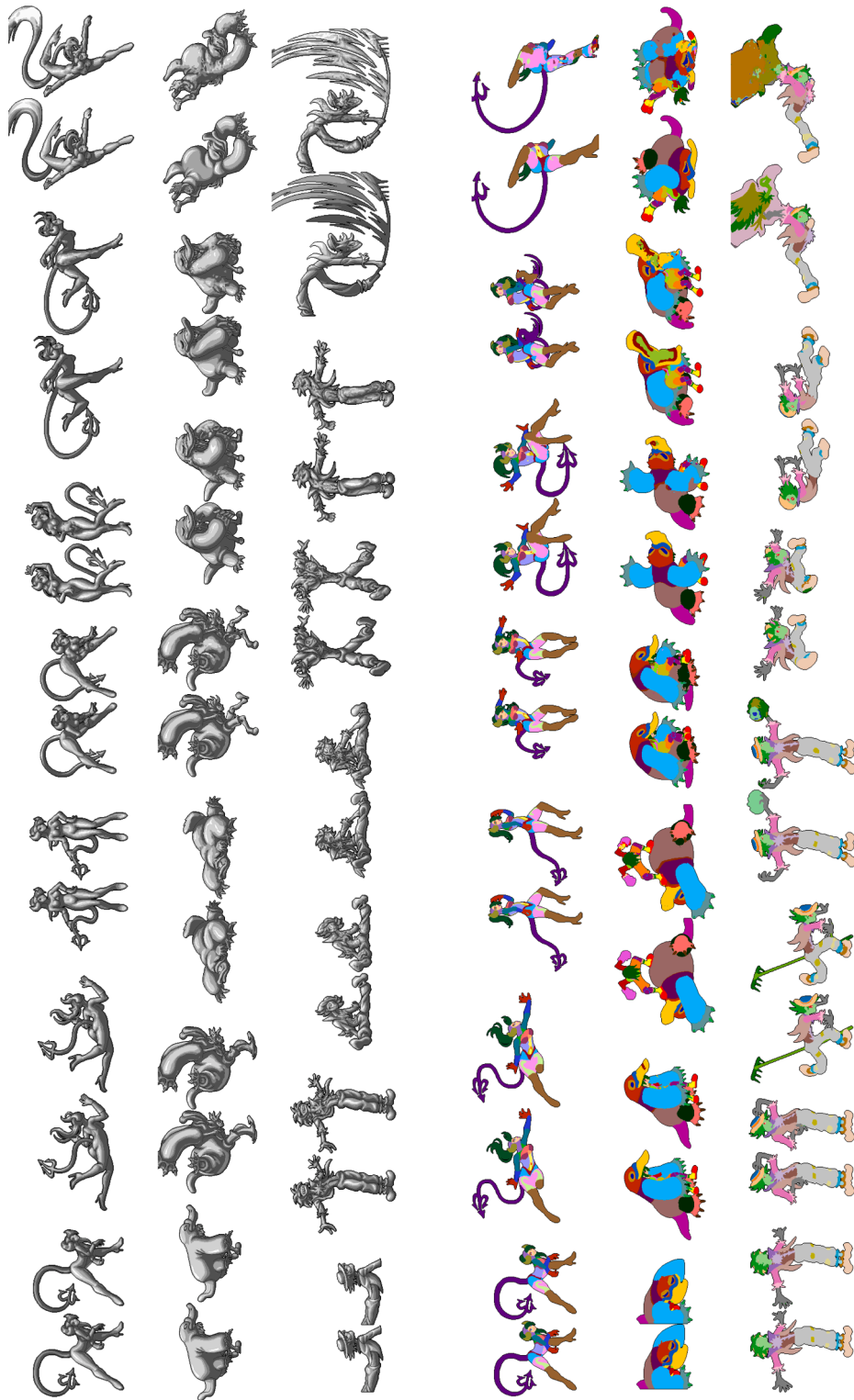


Fig. 7. Shading and coloring sprites generated by our solution for all three characters (*Lucy*, *Sarah*, and *Saulo*). From bottom-to-top, sprites are arranged from best to worst results and presented in pairs, with the artist drawn sprite to the left (ground truth) and the algorithm generated sprite to the right. All data collected from the test set.

similar fashion to shading sprites, although the obtained results have inaccuracies, several portions of each sprite could be used, mitigating the amount of work left for the human artist to complete.

Quantitatively, [Table 1](#) presents the accuracy, balanced accuracy, Top-3 and Top-5 metrics obtained in each task for each character. Considering shading sprites first, *Lucy* results are objectively the best

among all three characters, with 67% of pixels correctly classified, with *Saulo*'s being the worst, which correlates with our visual inspection. Comparing the standard accuracy and its balanced counterpart, the algorithm performed equally well on both metrics, indicating that its predictions are well balanced across every tone. Shifting to the Top-3 column, in almost every pixel, the algorithm was really close to or

Table 1

Quantitative results for all three characters on the shading and coloring problems.

	Model	Accuracy	Balanced	Top-3	Top-5
Shading	<i>Lucy</i>	67.37%	62.24%	95.55%	99.76%
	<i>Sarah</i>	65.93%	62.69%	94.84%	99.48%
	<i>Saulo</i>	58.80%	57.50%	92.45%	98.80%
Coloring	<i>Lucy</i>	88.11%	72.07%	97.69%	98.68%
	<i>Sarah</i>	83.32%	64.49%	94.85%	97.05%
	<i>Saulo</i>	83.61%	72.15%	96.04%	97.83%

landed on the right shading tone. Finally, regarding the Top-5 column, we included it for completeness sake, as it makes little to sense to evaluate it for a problem with six classes.

Looking at the last three rows of Table 1, which regards the coloring problem, we note a significant increase in accuracy. We attribute this boost to the nature of the problem: coloring each sprite region is mainly a recognition problem, not a creative one. Also, while the network has to account for shadows when considering shading, regions have little to no effect on each other when coloring is considered. Shifting to the balanced accuracy metric, results are significantly worse, as the class distribution for colors is long-tailed (Fig. 6). In practice, this means that the algorithm performs considerably better on more frequently seen colors than on more sparingly used ones. Nonetheless, the Top-3 and Top-5 metrics reveal that our algorithm is considerably close to the right answer in almost all pixels.

Fig. 8 shows the balanced accuracy evolution over the number of training epochs in the revised manuscript. For shading sprites, the model converges to its peak accuracy within forty epochs on all three characters. For color sprites, up to 160 passes over the dataset are needed to converge fully.

5.4. Ablation study

In this section, we present the results we obtained when developing our current model. In more detail, we present the relative improvement obtained from each element we introduced in our solution, starting with the base U-Net model. To streamline our investigation, we constrained this section to the evaluation of *Lucy* color sprites.

During our initial investigation, our main concern was to tackle the problems we found in our previous work, which were mainly on the coloring task [15]. For this, we kept our problem formulation as a pixel-to-pixel mapping problem, but we replaced the pixel-level task from a color regression problem to a full-fledged classification formulation, using one-hot encoded labels and the softmax activation function. This change directly addresses our issues with regressing color values for the coloring problem, which were challenging to quantize to the fixed set of tones used by each character. In Table 2, this new model corresponds to the first row of the table.

From this point forward, our goal was to encourage improving the algorithm performance. However, this proved quite challenging. Instead, considering human evaluation, we shifted our attention to steering our solution's errors to be less erroneous. For instance, it is

Table 2

Ablation Study considering the *Lucy* character color tones. Starting with the base U-Net model, each row adds a new element to the solution.

Model	Accuracy	Balanced	Top-3	Top-5	# Weights
Unet	87.35%	67.54%	96.98%	98.39%	61.7 M
w/ Dense Supervision	89.12%	70.30%	97.51%	98.64%	63.1 M
w/ Loss Weighing	89.38%	70.33%	97.74%	98.75%	-
w/ Focal Loss	88.87%	69.74%	97.67%	98.78%	-
w/ Class Weights	87.69%	71.33%	97.74%	98.85%	-
w/ Dense Connections	88.11%	72.07%	97.69%	98.68%	53.8 M

better to mistake the leg tone with the feet than with an eye or lip. For that reason, we implemented deep supervision, with the goal of encouraging the algorithm to focus its attention on a limited set of classes on each region by also supervising at lower resolutions. This change is shown at the second row of Table 2 and provided a noticeable improvement on all metrics. Marginal improvements were also obtained by individually weighting each loss term (third row of Table 2).

To further push the algorithm towards correctly classifying minority classes, we investigated the use of a class-weighted focal loss [19,20] (fourth and fifth rows of Table 2). These terms seek to even out class imbalance by re-weighting each class contribution to the final loss and by gradually assigning more attention to less confident predictions. In our experiments, they led to an additional percentage point in balanced accuracy, at the expense of some raw accuracy. We deem this trade-off worthy, as the accuracy we lost likely comes from majority classes while the gains we obtained come from less frequent labels. In practice, this means we traded more problematic errors for more forgiving ones.

Finally, in an effort to study if adding more depth to the network would yield additional benefit, we resorted to dense connections [17,18]. We chose this approach as it allows us to increase depth without compromising GPU resources. In practice, this regained most of the lost raw accuracy while also boosting our balanced one (last row of Table 2). However, these gains came at a cost. While the model shrank almost ten million parameters, its memory use and execution time nearly doubled, due to the increased number of layers, slowing down iteration times considerably.

5.5. Threats to validity

So far, we have concerned ourselves with the specific problems of *Trajes Fatais: Suits of Fate*'s pipeline, namely the problems of shading and coloring sprites. In our scenario, every character has a minimum of 300 sprites, with the average being 507. While this number is quite small in contrast to other problems [11,26], the majority of games use considerably less sprites per character, specially outside the fighting genre. This puts most game-related problems in the realm of few-shot learning [51] and, hence, our approach might not work or be the most sensible path of action. Additionally, the segmentation formulation might not be the most sensible approach for problems with a large number of output classes (e.g., 50 or more). Moreover, while it is not unusual for

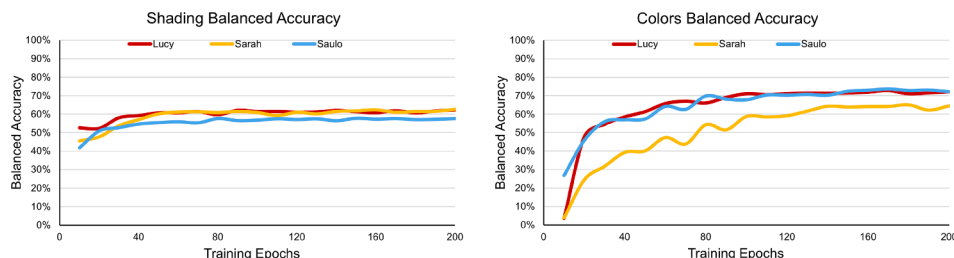


Fig. 8. Balanced accuracy evolution over training epochs for all three characters (*Lucy*, *Sarah*, and *Saulo*).

developers to adopt some approach to skin their characters, the approach taken by other developers might not translate well to how we approached the drawing task using neural networks.

Regarding our architectural choices, while Table 2 shows the contribution of each added element, the overall absolute improvement, in balanced accuracy terms, is 4.53 percentage points. While these results are valid in our case, on related problems, some of the included elements might perform better, worse or, in the worst case, adversely affect performance.

On a similar note, we evaluated the solution elements mainly under the balanced accuracy score, which attempts to summarize the entirety of our results as a single class balanced score. Under a closer inspection of the individual classes or under a broader/larger inspection, encompassing both tasks on all three (or more) characters, some elements might have proven to be more or less relevant than stated.

6. Conclusion and future work

In this work, we investigated the use of deep learning algorithms to create pixel art sprites from line art sketches to produce artwork that is of sufficient quality to be used within a game with little to no manual editing from human artists. More specifically, we set this work within the *Trajes Fatais: Suits of Fate* pipeline, which consists of drawing shading and coloring sprites from line-art sketches. This work extends our previous research [15] on the topic with a vastly improved model, embracing the problem as a pixel-wise classification task without adversarial training and presenting significantly improved results for both shading and coloring tasks. Ultimately, this work demonstrates positive evidence that the current image generation state-of-the-art is ready to be used in game-related tasks, especially within problems that lack large datasets, deviate considerably from natural images, and are extremely intolerant to errors and artifacts of any kind.

Future studies can follow at least three potential research directions: (1) further use cases of image generation within game pipelines, (2) the design of AI-assisted production schemes, and (3) enhancing the neural network design for improved sprite generation. In more detail, some worth exploring use cases of image generation in games are: to create in-between frames, to blend sprites, game style transfer, style-preserving rescaling routines, among others. Regarding AI-assisted pipelines, it is worth exploring how to leverage even smaller datasets, such as incomplete characters, and how to train such models incrementally using active learning and data-centric techniques [51]. Finally, there is plenty of room to improve the current neural network design. These are perceptual losses [54,55], the attention mechanism [44], and additional artistic control elements [11].

Finally, machine learning is showing promising footholds in creative art and making it more accessible. Nevertheless, despite its fundamental relevance, we believe that it will never completely wipe out the traditional forms of artistic creativity. On the contrary, we believe machines have the ultimate potential to unlock our most creative selves.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors would like to thank André, Onofre, and Anderson for the invaluable feedback on the generated artwork and Onanim Studios for granting access to the game's artwork.

References

- [1] P. Zackariasson, T.L. Wilson, *The video game industry: Formation, present state, and future*, Routledge, 2012.
- [2] E. Folmer, Component based game development. a solution to escalating costs and expanding deadlines?, in: *International Symposium on Component-Based Software Engineering* Springer, 2007, pp. 66–73.
- [3] J. Gregory, *Game engine architecture*, CRC Press, AK Peters, 2017.
- [4] N. Shaker, J. Togelius, M.J. Nelson, *Procedural content generation in games*, Springer, 2016.
- [5] R.R. Torrado, A. Khalifa, M.C. Green, N. Justesen, S. Risi, J. Togelius, Bootstrapping conditional gans for video game level generation, in: *2020 IEEE Conference on Games (CoG)*, IEEE, 2020, pp. 41–48.
- [6] V. Chaudhary, M. Sharma, P. Sharma, D. Agarwal, *Deep Learning in Gaming and Animations: Principles and Applications*, CRC Press, 2021.
- [7] J. Liu, S. Snodgrass, A. Khalifa, S. Risi, G.N. Yannakakis, J. Togelius, Deep learning for procedural content generation, *Neural Comput. Appl.* (2020) 1–19.
- [8] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Informat. Process. Syst.* 25 (2012) 1097–1105.
- [9] D.P. Kingma, M. Welling, Auto-encoding variational bayes, *arXiv preprint arXiv: 1312.6114* (2013) 1–14.
- [10] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, in: *Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680.
- [11] T. Karras, S. Laine, T. Aila, A style-based generator architecture for generative adversarial networks, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4401–4410.
- [12] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444.
- [13] Y.R. Serpa, L.A. Pires, M.A.F. Rodrigues, Milestones and new frontiers in deep learning, in: *2019 32nd SIBGRAPI Conference on Graphics, Patterns and Images Tutorials (SIBGRAPI-T)*, IEEE, 2019, pp. 22–35. Available at: <https://ieeexplore.ieee.org/document/8920294>.
- [14] P. Isola, J.-Y. Zhu, T. Zhou, A.A. Efros, Image-to-image translation with conditional adversarial networks, *IEEE*, 2017, pp. 1125–1134.
- [15] Y.R. Serpa, M.A.F. Rodrigues, Towards machine-learning assisted asset generation for games: A study on pixel art sprite sheets, in: *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, IEEE, 2019, pp. 182–191. Available at: <https://doi.ieeecomputersociety.org/10.1109/SBGames.2019.00032>.
- [16] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: *Proceedings of the International Conference on Medical image computing and computer-assisted intervention*, Springer, 2015, pp. 234–241.
- [17] N. Irbetaz, M.S. Rahman, Multiresunet: Rethinking the u-net architecture for multimodal biomedical image segmentation, *Neural Networks* 121 (2020) 74–87.
- [18] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4700–4708.
- [19] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [20] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, S. Belongie, Class-balanced loss based on effective number of samples, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 9268–9277.
- [21] H. Huang, L. Lin, R. Tong, H. Hu, Q. Zhang, Y. Iwamoto, X. Han, Y.-W. Chen, J. Wu, Unet 3+: A full-scale connected unet for medical image segmentation, in: *ICASSP 2020–2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2020, pp. 1055–1059.
- [22] D. Berthelot, T. Schumm, L. Metz, Began: Boundary equilibrium generative adversarial networks, *arXiv preprint arXiv:1703.10717*.
- [23] A. Srivastava, L. Valkov, C. Russell, M.U. Gutmann, C. Sutton, Veegan: Reducing mode collapse in gans using implicit variational learning, *Adv. Neural Informat. Process. Syst.* (2017) 3308–3318.
- [24] Z. Zhang, M. Li, J. Yu, On the convergence and mode collapse of GAN, in: *SIGGRAPH Asia 2018 Technical Briefs*, ACM, 2018, pp. 1–18.
- [25] T. Karras, T. Aila, S. Laine, J. Lehtinen, Progressive growing of gans for improved quality, stability, and variation, *arXiv preprint arXiv:1710.10196*.
- [26] A. Brock, J. Donahue, K. Simonyan, Large scale gan training for high fidelity natural image synthesis, *arXiv preprint arXiv:1809.11096*.
- [27] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, T. Aila, Analyzing and improving the image quality of stylegan, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 8110–8119.
- [28] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, T. Aila, Alias-free generative adversarial networks, *Adv. Neural Informat. Process. Syst.* 34.
- [29] L.A. Gatys, A.S. Ecker, M. Bethge, Image style transfer using convolutional neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2016, pp. 2414–2423.
- [30] S. Iizuka, H. Simo-Serra, H. Ishikawa, Let there be color!: joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification, *ACM Trans. Graphics* 35 (4) (2016) 1–11.
- [31] J.-Y. Zhu, T. Park, P. Isola, A.A. Efros, Unpaired image-to-image translation using cycle-consistent adversarial networks, in: *Proceedings of the IEEE International Conference on Computer Vision*, IEEE, 2017, pp. 2223–2232.
- [32] H. Li, Y. Yang, M. Chang, S. Chen, H. Feng, Z. Xu, Q. Li, Y. Chen, Srdiff: Single image super-resolution with diffusion probabilistic models, *Neurocomputing*, (2022).

- [33] M. McGuire, M. Gagliu, Mpx style-preserving pixel-art magnification, *J. Comput. Graphics Tech.* 10 (2) (2021).
- [34] G. Lample, N. Zeghidour, N. Usunier, A. Bordes, L. Denoyer, et al., Fader networks: Manipulating images by sliding attributes, *Adv. Neural Informat. Process. Syst.* (2017) 5967–5976.
- [35] T. Xue, J. Wu, K. Bouman, B. Freeman, Visual dynamics: Probabilistic future frame synthesis via cross convolutional networks, *Adv. Neural Informat. Process. Syst.* (2016) 91–99.
- [36] Y. Chen, Y.-K. Lai, Y.-J. Liu, Cartoongan: Generative adversarial networks for photo cartoonization, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 9465–9474.
- [37] Y. Liu, Z. Qin, T. Wan, Z. Luo, Auto-painter: Cartoon image generation from sketch by using conditional wasserstein generative adversarial networks, *Neurocomputing* 311 (2018) 78–87.
- [38] Y. Jin, J. Zhang, M. Li, Y. Tian, H. Zhu, Z. Fang, Towards the automatic anime characters creation with generative adversarial networks, *arXiv preprint arXiv:1708.05509*.
- [39] G. Liu, X. Chen, Y. Hu, Anime sketch coloring with swish-gated residual u-net, in: *International Symposium on Intelligence Computation and Applications*, Springer, 2018, pp. 190–204.
- [40] A. Vahdat, J. Kautz, Nvae: A deep hierarchical variational autoencoder, *Adv. Neural Informat. Process. Syst.* 33 (2020) 19667–19679.
- [41] T. Salimans, A. Karpathy, X. Chen, D.P. Kingma, Pixcnn++: Improving the pixcnn with discretized logistic mixture likelihood and other modifications, *arXiv preprint arXiv:1701.05517*.
- [42] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, I. Sutskever, Zero-shot text-to-image generation, in: *International Conference on Machine Learning*, PMLR, 2021, pp. 8821–8831.
- [43] M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, I. Sutskever, Generative pretraining from pixels, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 1691–1703.
- [44] H. Zhang, I. Goodfellow, D. Metaxas, A. Odena, Self-attention generative adversarial networks, in: *International conference on machine learning*, PMLR, 2019, pp. 7354–7363.
- [45] K. Lee, H. Chang, L. Jiang, H. Zhang, Z. Tu, C. Liu, Vitgan: Training gans with vision transformers, *arXiv preprint arXiv:2107.04589*.
- [46] Y. Jiang, S. Chang, Z. Wang, Transgan: Two pure transformers can make one strong gan, and that can scale up, *Adv. Neural Informat. Process. Syst.* 34 (2021).
- [47] L. Horsley, D. Perez-Liebana, Building an automatic sprite generator with deep convolutional generative adversarial networks, in: *Proceedings of the IEEE Conference on Computational Intelligence and Games*, IEEE, 2017, pp. 134–141.
- [48] D. Smirnov, M. Gharbi, M. Fisher, V. Guizilini, A. Efros, J.M. Solomon, Marionette: Self-supervised sprite learning, *Adv. Neural Informat. Process. Syst.* 34 (2021).
- [49] E. Chen, C. Sydora, B. Burega, A. Mahajan, A. Abdullah, M. Gallivan, M. Guzdial, Image-to-level: Generation and repair, in: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 16, 2020, pp. 189–195.
- [50] A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al., Learning transferable visual models from natural language supervision, *arXiv preprint arXiv:2103.00020*.
- [51] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444.
- [52] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *International Conference on Machine Learning*, PMLR, 2015, pp. 448–456.
- [53] Google, Tensorflow: A system for large-scale machine learning, in: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 265–283.
- [54] J. Johnson, A. Alahi, L. Fei-Fei, Perceptual losses for real-time style transfer and super-resolution, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2016, pp. 694–711.
- [55] R. Zhang, P. Isola, A.A. Efros, E. Shechtman, O. Wang, The unreasonable effectiveness of deep features as a perceptual metric, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2018, pp. 586–595.